Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет инженерно-экономический

Кафедра экономической информатики

ЛАБОРАТОРНАЯ РАБОТА № 2
по теме:
Юнит-тестирование в Spring Boot с использованием JUnit 5 и Mockito

Выполнил — Зуёнок И.А.
студент группы 377901

Проверил — М.Р. Ахриева,
ассистент кафедры ЭИ

Минск 2026

Лабораторная работа № 2

«Юнит-тестирование в Spring Boot с использованием JUnit 5 и Mockito»

Цель работы: научиться писать юнит-тесты для приложения Spring Boot, используя JUnit 5 и Mockito.

Листинг кода:

Класс MovieRepositoryTest:

```
@DataJpaTest
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class MovieRepositoryTest {

    @Autowired
    private MovieRepository movieRepository;

    private Movie movie;

    @BeforeEach
    public void setUp() {
        movie = new Movie();
        movie.setSlug("avengers-endgame");
        movie.setTitle("Мстители: Финал");
        movie.setDescription("Эпичное завершение саги Marvel.");
        movie.setReleaseYear(2019);
        movie.setGenre("Фантастика");
        movie.setDurationMinutes(181);
    }

    @Test
    @Order(1)
    @DisplayName("Сохранение фильма")
    @Rollback(value = false)
    public void saveMovieTest() {
        Movie saved = movieRepository.save(movie);

        Assertions.assertThat(saved.getId()).isGreaterThan(0);
        Assertions.assertThat(saved.getTitle()).isEqualTo("Мстители:
Финал");
    }

    @Test
    @Order(2)
    @DisplayName("Поиск фильма по slug")
    public void findBySlugTest() {
        movieRepository.save(movie);

        Optional<Movie> found = movieRepository.findBySlug("avengers-
endgame");

        Assertions.assertThat(found).isPresent();
        Assertions.assertThat(found.get().getTitle()).isEqualTo("Мстители:
Финал");
    }

    @Test
    @Order(3)
```

```
    @DisplayName("Список всех фильмов")
    public void findAllTest() {
        movieRepository.save(movie);

        List<Movie> movies = movieRepository.findAll();

        Assertions.assertThat(movies).isNotEmpty();
        Assertions.assertThat(movies.size()).isGreaterThan(0);
    }

    @Test
    @Order(4)
    @DisplayName("Обновление фильма")
    @Rollback(value = false)
    public void updateMovieTest() {
        Movie saved = movieRepository.save(movie);
        saved.setTitle("Мстители: Финал (обновлено)");

        Movie updated = movieRepository.save(saved);

        Assertions.assertThat(updated.getTitle()).isEqualTo("Мстители:
Финал (обновлено)");
    }

    @Test
    @Order(5)
    @DisplayName("Удаление фильма")
    @Rollback(value = false)
    public void deleteMovieTest() {
        Movie saved = movieRepository.save(movie);
        Long id = saved.getId();

        movieRepository.deleteById(id);

        Assertions.assertThat(movieRepository.findById(id)).isEmpty();
    }
}
```

Класс MoviesControllerUnitTest:

```
@WebMvcTest(MoviesController.class)
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class MoviesControllerUnitTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private MovieService movieService;

    @Autowired
    private ObjectMapper objectMapper;

    private MovieDTO movieDTO;
    private MovieCreateDTO createDTO;

    @BeforeEach
    public void setUp() {
        movieDTO = new MovieDTO();
        movieDTO.setId(1L);
        movieDTO.setSlug("avengers-endgame");
        movieDTO.setTitle("Мстители: Финал");
```

```
            movieDTO.setDescription("Эпичное завершение саги Marvel.");
            movieDTO.setReleaseYear(2019);
            movieDTO.setGenre("Фантастика");
            movieDTO.setDurationMinutes(181);
            movieDTO.setCreatedAt(LocalDate.now());

            createDTO = new MovieCreateDTO();
            createDTO.setSlug("avengers-endgame");
            createDTO.setTitle("Мстители: Финал");
            createDTO.setDescription("Эпичное завершение саги Marvel.");
            createDTO.setReleaseYear(2019);
            createDTO.setGenre("Фантастика");
            createDTO.setDurationMinutes(181);
        }

        @Test
        @Order(1)
        @DisplayName("POST /api/movies — создание фильма")
        public void createMovieTest() throws Exception {

given(movieService.create(any(MovieCreateDTO.class))).willReturn(movieDTO);

            ResultActions response = mockMvc.perform(post("/api/movies")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(createDTO)));

            response.andDo(print())
                    .andExpect(status().isCreated())
                    .andExpect(jsonPath("$.title", is(movieDTO.getTitle())))
                    .andExpect(jsonPath("$.slug", is(movieDTO.getSlug())))
                    .andExpect(jsonPath("$.genre", is(movieDTO.getGenre())));
        }

        @Test
        @Order(2)
        @DisplayName("GET /api/movies — список фильмов")
        public void getAllMoviesTest() throws Exception {
            MovieDTO second = new MovieDTO();
            second.setId(2L);
            second.setSlug("inception");
            second.setTitle("Начало");
            second.setGenre("Фантастика");

            given(movieService.getAll()).willReturn(List.of(movieDTO, second));

            ResultActions response = mockMvc.perform(get("/api/movies"));

            response.andExpect(status().isOk())
                    .andDo(print())
                    .andExpect(jsonPath("$.size()", is(2)))
                    .andExpect(jsonPath("$[0].title", is("Мстители: Финал")))
                    .andExpect(jsonPath("$[1].title", is("Начало")));
        }

        @Test
        @Order(3)
        @DisplayName("GET /api/movies/{id} — фильм по ID")
        public void getMovieByIdTest() throws Exception {
            given(movieService.getById(1L)).willReturn(movieDTO);

            ResultActions response = mockMvc.perform(get("/api/movies/{id}",
1L));

            response.andExpect(status().isOk())
```

```
                    .andDo(print())
                    .andExpect(jsonPath("$.title", is(movieDTO.getTitle())))
                    .andExpect(jsonPath("$.slug", is(movieDTO.getSlug())));
        }

        @Test
        @Order(4)
        @DisplayName("PUT /api/movies/{id} — обновление фильма")
        public void updateMovieTest() throws Exception {
            var updateDTO = new MovieUpdateDTO();
            movieDTO.setTitle("Мстители: Финал (обновлено)");

            given(movieService.update(eq(1L),
any(MovieUpdateDTO.class))).willReturn(movieDTO);

            ResultActions response = mockMvc.perform(put("/api/movies/{id}",
1L)
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(updateDTO)));

            response.andExpect(status().isOk())
                    .andDo(print())
                    .andExpect(jsonPath("$.title", is("Мстители: Финал
(обновлено)")));
        }

        @Test
        @Order(5)
        @DisplayName("DELETE /api/movies/{id} — удаление фильма")
        public void deleteMovieTest() throws Exception {
            willDoNothing().given(movieService).delete(1L);

            ResultActions response = mockMvc.perform(delete("/api/movies/{id}",
1L));

            response.andExpect(status().isNoContent())
                    .andDo(print());
        }
    }
```

Класс MovieServiceUnitTest:

```
    @ExtendWith(MockitoExtension.class)
    @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
    public class MovieServiceUnitTest {

        @Mock
        private MovieRepository movieRepository;

        @Mock
        private MovieMapper movieMapper;

        @InjectMocks
        private MovieService movieService;

        private Movie movie;
        private MovieDTO movieDTO;

        @BeforeEach
        public void setUp() {
            movie = new Movie();
            movie.setId(1L);
```

```
        movie.setSlug("avengers-endgame");
        movie.setTitle("Мстители: Финал");
        movie.setDescription("Эпичное завершение саги Marvel.");
        movie.setReleaseYear(2019);
        movie.setGenre("Фантастика");
        movie.setDurationMinutes(181);

        movieDTO = new MovieDTO();
        movieDTO.setId(1L);
        movieDTO.setSlug("avengers-endgame");
        movieDTO.setTitle("Мстители: Финал");
        movieDTO.setGenre("Фантастика");
    }

    @Test
    @Order(1)
    @DisplayName("getAll — возвращает список фильмов")
    public void getAllTest() {
        Movie second = new Movie();
        second.setId(2L);
        second.setTitle("Начало");

        given(movieRepository.findAll()).willReturn(List.of(movie,
second));
        given(movieMapper.map(movie)).willReturn(movieDTO);

        List<MovieDTO> result = movieService.getAll();

        assertThat(result).isNotNull();
        assertThat(result).isNotEmpty();
        verify(movieRepository, times(1)).findAll();
    }

    @Test
    @Order(2)
    @DisplayName("getById — возвращает фильм")
    public void getByIdTest() {
        given(movieRepository.findById(1L)).willReturn(Optional.of(movie));
        given(movieMapper.map(movie)).willReturn(movieDTO);

        MovieDTO result = movieService.getById(1L);

        assertThat(result).isNotNull();
        assertThat(result.getTitle()).isEqualTo("Мстители: Финал");
        verify(movieRepository, times(1)).findById(1L);
    }

    @Test
    @Order(3)
    @DisplayName("getById — бросает исключение, если фильма нет")
    public void getByIdNotFoundTest() {
        given(movieRepository.findById(99L)).willReturn(Optional.empty());

        assertThatThrownBy(() -> movieService.getById(99L))
                .isInstanceOf(ResourceNotFoundException.class)
                .hasMessageContaining("Movie not found");
    }

    @Test
    @Order(4)
    @DisplayName("create — сохраняет фильм")
    public void createTest() {
        given(movieMapper.map(any())).willReturn(movie);
        given(movieRepository.save(movie)).willReturn(movie);
```

```
        given(movieMapper.map(movie)).willReturn(movieDTO);

        MovieDTO result = movieService.create(any());

        assertThat(result).isNotNull();
        assertThat(result.getTitle()).isEqualTo("Мстители: Финал");
        verify(movieRepository, times(1)).save(movie);
    }

    @Test
    @Order(5)
    @DisplayName("update — обновляет фильм")
    public void updateTest() {
        given(movieRepository.findById(1L)).willReturn(Optional.of(movie));
        given(movieRepository.save(movie)).willReturn(movie);
        given(movieMapper.map(movie)).willReturn(movieDTO);

        MovieDTO result = movieService.update(1L, any());

        assertThat(result).isNotNull();
        verify(movieRepository, times(1)).save(movie);
    }

    @Test
    @Order(6)
    @DisplayName("update — бросает исключение, если фильма нет")
    public void updateNotFoundTest() {
        given(movieRepository.findById(99L)).willReturn(Optional.empty());

        assertThatThrownBy(() -> movieService.update(99L, any()))
                .isInstanceOf(ResourceNotFoundException.class);
    }

    @Test
    @Order(7)
    @DisplayName("delete — удаляет фильм")
    public void deleteTest() {
        willDoNothing().given(movieRepository).deleteById(1L);

        movieService.delete(1L);

        verify(movieRepository, times(1)).deleteById(1L);
    }
}
```

Скриншоты выполнения программы:

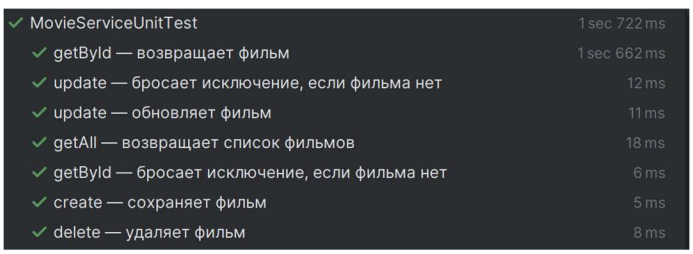


Рисунок 1 – MovieServiceUnitTest

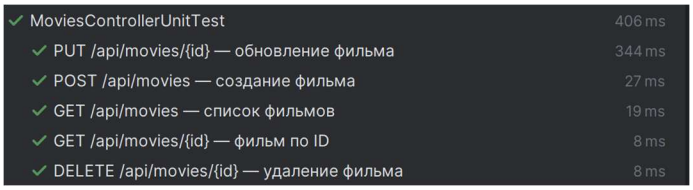


Рисунок 2 – MoviesControllerUnitTest



Рисунок 3 – MovieRepositoryTest

Контрольные вопросы:

1. Что такое JUnit?

Фреймворк для модульного тестирования Java-кода. Позволяет писать и запускать тесты, проверять ожидаемые результаты, организовывать тесты в наборы.

2. Основные аннотации JUnit: @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll, @DisplayName, @Disabled, @Nested, @ParameterizedTest, @RepeatedTest, @Timeout, @Tag, @ExtendWith.

3. Чем отличается JUnit 4 от JUnit 5?

JUnit 5 состоит из трёх модулей (Platform, Jupiter, Vintage), поддерживает Java 8+, новые аннотации (@BeforeEach вместо @Before), расширения через @ExtendWith вместо @RunWith, лучшую поддержку parameterized-тестов и параллельного запуска. JUnit 4 – монолитный, устаревший.

4. @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll: @Test – помечает тестовый метод. @BeforeEach – выполняется перед каждым тестом. @AfterEach – после каждого теста. @BeforeAll – один раз перед всеми тестами (static). @AfterAll – один раз после всех тестов (static).

5. Разница @BeforeEach и @BeforeAll:

@BeforeEach – перед каждым тестом (создание свежих данных).

@BeforeAll – один раз перед всем классом (тяжёлые ресурсы: подключение к БД). @BeforeAll должен быть static (если не используется @TestInstance(PER_CLASS)).

6. Assertions в JUnit 5: assertEquals, assertNotEquals, assertTrue, assertFalse, assertNull, assertNotNull, assertThrows, assertDoesNotThrow, assertAll, assertArrayEquals, assertIterableEquals, assertTimeout, assertSame, assertNotSame.

7. @DisplayName и @Disabled: @DisplayName – задаёт читаемое имя теста/класса для отчётов. @Disabled – отключает тест или класс (не выполняется).

8. @Nested: Позволяет группировать тесты во вложенные классы для

логической структуризации (например, по сценариям). Внутренние классы не static, могут иметь свои @BeforeEach.

9. Parameterized tests: Тесты, которые выполняются несколько раз с разными входными данными. Пишутся через @ParameterizedTest + источник данных.

10. @ParameterizedTest и источники: @ValueSource – простые значения (int, String). @CsvSource – CSV-строки. @MethodSource – метод-поставщик Stream<Arguments>. Также: @EnumSource, @CsvFileSource, @ArgumentsSource.

11. @RepeatedTest: Повторяет тест N раз. Пример: @RepeatedTest(5).

12. @Timeout: Ограничивает время выполнения теста. Если превышено – тест падает. Пример: @Timeout(2) – 2 секунды.

13. assertThrows: Проверяет, что метод выбрасывает ожидаемое исключение: java assertThrows(IllegalArgumentException.class, () -> service.doStuff());

14. @Tag: Метка для категоризации тестов (например, @Tag("slow"), @Tag("integration")). Позволяет фильтровать запуск.

15. Test Lifecycle и порядок: По умолчанию порядок методов не гарантирован. Можно задать @TestMethodOrder (@Order, @Random, @MethodName). Жизненный цикл: @BeforeAll → @BeforeEach → @Test → @AfterEach → @AfterAll.

16. @TestInstance: Управляет жизненным циклом экземпляра теста: PER_METHOD (по умолчанию) – новый экземпляр на каждый тест. PER_CLASS – один экземпляр на класс (позволяет не-static @BeforeAll).

17. @ExtendWith и расширения: Регистрирует расширения JUnit 5 (аналог @RunWith в JUnit 4). Пример: @ExtendWith(MockitoExtension.class), @ExtendWith(SpringExtension.class).

18. Assumptions vs assertions: Assumptions (assumeTrue, assumeFalse) – если условие не выполнено, тест пропускается (не падает). Assertions – если не выполнено, тест падает.

19. @TempDir: Создаёт временную директорию для теста, автоматически удаляется после. Удобно для тестов работы с файлами.

20. Параллельный запуск: В junit-platform.properties: text junit.jupiter.execution.parallel.enabled=true
junit.jupiter.execution.parallel.mode.default=concurrent Mockito

21. Mockito: Библиотека для создания mock-объектов и тестирования взаимодействий. Позволяет подменять зависимости, задавать поведение и проверять вызовы.

22. mock, spy, stub, fake: mock – полностью поддельный объект, все методы возвращают дефолт, пока не задано поведение. spy – обёртка над реальным объектом, вызывает реальные методы, если не переопределено. stub – объект с заранее заданными ответами (mock с настроенным поведением). fake – упрощённая рабочая реализация (например, in-memory репозиторий).

23. @Mock, @Spy, @InjectMocks: @Mock – создаёт mock. @Spy – создаёт spy. @InjectMocks – создаёт объект и внедряет в него @Mock/@Spy поля.

24. @MockBean vs @Mock: @MockBean – Spring-аннотация, заменяет бин

в контексте Spring (для интеграционных тестов). @Mock – чисто Mockito, не связан со Spring-контекстом.

25. when(...).thenReturn(...) и stubbing: Задаёт поведение mock-объекта: java when(repo.findById(1L)).thenReturn(Optional.of(user));

26. doReturn(...).when(...) vs when(...).thenReturn(...): doReturn().when() безопаснее для spy и void-методов, не вызывает реальный метод. when().thenReturn() вызывает реальный метод при настройке spy – может привести к нежелательным эффектам.

27. verify(): Проверяет, что метод был вызван: java verify(repo).save(user); verify(repo, times(2)).findById(any());

28. ArgumentMatchers: any(), anyString(), anyInt(), eq(), isNull(), isNotNull(), contains(), startsWith(), argThat().

29. any(), eq(), anyString(), isNull(): any() – любой аргумент (включая null). eq(value) – точное совпадение. anyString() – любая не-null строка. isNull() – null.

30. verifyNoMoreInteractions() и verifyNoInteractions(): verifyNoMoreInteractions(mock) – после всех verify не осталось непроверенных вызовов. verifyNoInteractions(mock) – с mock вообще не взаимодействовали. 31. ArgumentCaptor: Захватывает аргументы вызова для последующей проверки: java ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class); verify(repo).save(captor.capture()); assertEquals("John", captor.getValue().getName());

32. doNothing(), doThrow(), doAnswer(): doNothing() – ничего не делать (для void). doThrow() – выбросить исключение. doAnswer() – кастомная логика ответа.

33. @Captor: Аннотация для автоматического создания ArgumentCaptor: java @Captor ArgumentCaptor<User> captor;

34. MockitoAnnotations.openMocks() и @ExtendWith(MockitoExtension.class): Оба инициализируют mock-аннотации. openMocks() – вручную в @BeforeEach. @ExtendWith(MockitoExtension.class) – автоматически, современный способ.

35. lenient() и strict stubs: По умолчанию Mockito strict – падает, если stub не использован. lenient() отключает проверку для конкретного stub. Strict stubs помогают находить лишние настройки.

36. Мок статических методов: Через mockStatic() (Mockito 3.4+): java try (MockedStatic<Utils> mocked = mockStatic(Utils.class)) { mocked.when(() -> Utils.foo()).thenReturn("bar"); }

37. Мок final-классов и методов: Встроенный mock-maker-inline (Mockito 2+) – работает из коробки. Для старых версий нужен mockito-inline артефакт.

38. thenAnswer() vs thenReturn(): thenReturn(value) – возвращает фиксированное значение. thenAnswer(invocation -> ...) – вычисляет ответ динамически на основе аргументов.

39. InOrder: Проверяет порядок вызовов: java InOrder inOrder = inOrder(a, b); inOrder.verify(a).foo(); inOrder.verify(b).bar();

40. reset(): Сбрасывает mock к исходному состоянию (удаляет stubs и историю). Использовать редко – обычно признак плохого дизайна теста. Виды тестов

41. Unit-тест: Тестирует отдельный класс/метод в изоляции, все зависимости замоканы. Быстрый.

42. Интеграционный тест: Проверяет взаимодействие нескольких компонентов (например, сервис + БД, контроллер + сервис).

43. Unit vs интеграционный: Unit – изолированно, быстро, mock-зависимости. Интеграционный – реальные компоненты, медленнее, проверяет связки.

44. Компонентный тест: Тестирует отдельный компонент системы целиком (например, весь модуль), но без внешних систем.

45. E2E-тест: Проверяет всю систему от начала до конца (UI → backend → БД), имитируя реального пользователя.

46. Smoke-тест: Быстрая проверка, что основная функциональность работает после сборки/деплоя.

47. Регрессионный тест: Проверяет, что новые изменения не сломали существующую функциональность.

48. Тестовая пирамида: Много unit-тестов (основание), меньше интеграционных (середина), мало E2E (вершина). Быстрота и стоимость растут к вершине.

49. TDD: Test-Driven Development: сначала пишем падающий тест → код → рефакторинг. Цикл Red-Green-Refactor.

50. BDD vs TDD: BDD (Behavior-Driven Development) – тесты описывают поведение на бизнес-языке (Given-When-Then), часто через Cucumber. TDD – про техническую реализацию. Spring Test

51. @SpringBootTest: Загружает полный Spring-контекст для интеграционных тестов. Может поднимать весь app или отдельные слои.

52. @WebMvcTest: Тестирует только web-слой (контроллеры), не загружает сервисы и репозитории. Требует @MockBean для зависимостей. 53. @DataJpaTest: Тестирует JPA-слой (репозитории), поднимает только JPA-контекст с in-memory БД (H2 по умолчанию).

54. @RestClientTest: Тестирует REST-клиенты (RestTemplate, WebClient) с mock-сервером.

55. Testcontainers: Библиотека для запуска реальных Docker-контейнеров (PostgreSQL, Kafka и т.д.) в тестах. Даёт реалистичное окружение.

56. In-memory БД (H2): Быстрая БД в памяти для тестов. Используется когда реальная СУБД не важна, а нужна скорость. Минус – отличается от продакшн-БД.

57. Mocking в интеграционных тестах: Применяется для внешних систем (платёжные шлюзы, сторонние API), чтобы не зависеть от них. Внутренние компоненты лучше не мокать – иначе тест теряет смысл.

58. Test coverage и метрики: Процент кода, покрытого тестами. Метрики: line, branch, method, class. Инструменты: JaCoCo, Cobertura. 100% покрытие ≠ отсутствие багов.

59. Mutation testing: В код вносятся мелкие мутации (изменение оператора, удаление строки). Если тесты не падают – они слабые. Инструмент: PIT.

60. Flaky test: Тест, который иногда проходит, иногда падает без

изменений в коде. Причины: гонки, время, порядок, внешние зависимости. Борьба: изоляция, фиксированные данные, retry, устранение недетерминизма.

Вывод:

В ходе лабораторной работы я научилась писать юнит-тесты для приложения Spring Boot, используя JUnit 5 и Mockito.